

PARALLEL RISC PROCESSOR

Ilya Kirshtein

Aim of the presentation

My goal is to introduce you to the world of processors by showing you my custom made RISC processor, whether you are a programmer, or simply a tech enthusiast.

This presentation will provide a clear glimpse into how RISC processors work under the hood.

The slides will cover the architecture, key components, and their roles within the CPU. I will also demonstrate a few simple and visual examples of the processor in action to make the concepts easier to understand.

Disclaimer: This processor is heavily inspired by RISC architecture, but some components are custom made, This means that Modern / Professional processors might differ, I will note throughout the slides the differences I know of.

Project Overview

RISC – reduced instruction set computer

A popular architecture for microcontrollers and embedded systems

The key operational concept of the RISC computer is that each instruction performs only one function.

This simplicity allows for faster instruction execution, efficient pipelining, and reduced hardware complexity, making RISC ideal for high-performance and low-power applications.

This Parallel RISC Processor demonstrates the principles of RISC-based CPU design, pipelining, hazard management, and parallel instruction execution. Through custom VHDL implementation, dual-ALU architecture.

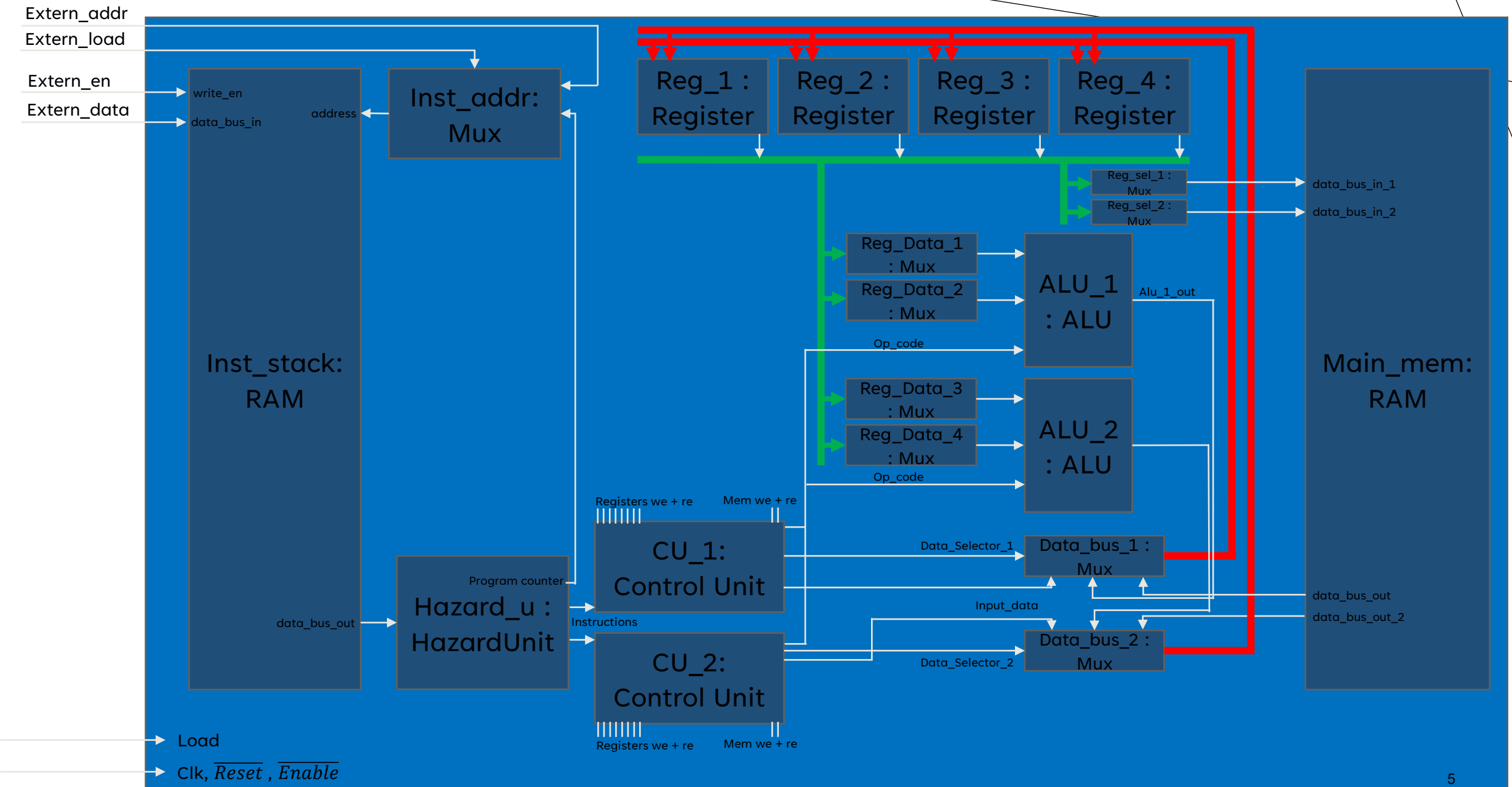
Parallel Execution

The processor is designed with two independent instruction pipelines, allowing it to process up to two instructions per clock cycle. In this presentation, we will explore how this mechanism works, whether it truly improves performance, and the potential challenges and risks involved when attempting to execute two instructions simultaneously.

Components:

- RAM – Main working memory, used to read and write data during run time.
The CPU includes 2 of those in different ‘costumes’ one that works as explained above, and one that works as read only stack for the instructions.
- Register – High speed memory unit, capable of storing a single data entry, connected directly to the ALU.
Our Processor provides 4 Registers, R1, R2, R3, R4.
- ALU – Arithmetic Logic Unit, Responsible for logical and mathematical operations, receives data only through registers.
- Hazard Unit – Analyzes the instruction stack for instruction conflicts and takes steps to prevent logical errors.
- Control Unit – Orchestrator of the CPU pipeline. On each clock cycle takes the next instruction, breaks it down and signals relevant components for execution.
- Multiplexer (Mux) – An electrical switch, has few inputs, and an external selector that chooses which input to forward to the output.

Overall picture: don't worry, we will explore this part by part later on.



Instructions breakdown

Our processor uses 16bit instructions that he break down and understands the following way:

For ALU operations

XXXX	XXXX	XXXX	XXXX
↑	↑	↑	↑
OP	dest	src1	src2

For memory operations

XXXX	XXXX	XXXXXXXXXX
↑	↑	↑
OP	dest	Addr/value

RISC utilizes **load-store architecture** meaning that instructions are divided into two categories: memory access and ALU operations that only occur between registers.

OP – operation code, the full list available next slide.

Dest – target of the operation, can only be a register.

Src1/2 – operands, can only be registers.

Addr – memory address in the RAM

Value – an immediate value to load into a register.

Instructions breakdown

Available instructions:

Instruction	Binary
NOP – no op, ‘do nothing’	0000
LOAD – load from memory to register	0001
STORE – save data from register to memory	0010
LOADI – load an immediate value to a register	0011
ADD – Add 2 registers into a register	0100
SUB – subtract 2 registers	0101
MUL – multiply 2 registers	0110
OR – logical OR between registers	0111

AND – logical AND	1000
XOR – logical XOR	1001
SHL – shift left	1010
SHR – shift right	1011
CMP – compare 2 registers, turns on the ZERO_FLAG	1100
MOV – moves data from one register to another	1101
JMP – jump to another instruction	1110
JE – jump only if ZERO_FLAG is on by the previous instruction	1111

*Note: classical RISC CPUs are usually 32bit, meaning they support way more instructions.

Registers

As discussed before, registers is our main working unit, almost every operation has something to do with a register. Some things worth mentioning:

In the diagram above I showed 4 registers, but that's not the entire truth, there are many registers around the CPU, but not all of them are available to the programmer, for example:

Flags register – to track various flags, zero_flag, overflow_flag, sign_flag.

Instruction Counter – points to the current instruction on the stack.

Modern RISC processors may have many utility registers that could be: timers, loop counters, communication with peripheral devices, etc...

Some RISC architectures choose to have static registers for each ALU, for example R1 and R2 are always connected to ALU_1 and only to it.

I made an arbitrary choice of 4 free use registers.

CPU Pipeline

The CPU uses a four stage pipeline and schedules instructions between ALU_1 and ALU_2.

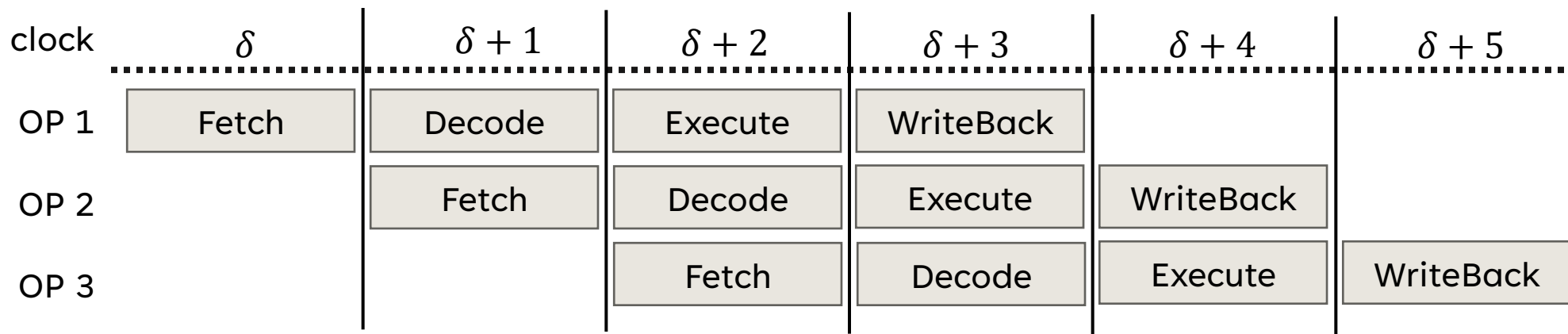
If two instructions are independent, they execute in parallel.

If a data hazard is detected, the Hazard Unit issues NOPs to maintain correctness.

- Fetch – The next instructions get fetched from the Hazard Unit by the Control Unit.
- Decode – The Control Unit breaks the instruction down and gets the relevant components ready
- Execute – Instruction logic gets executed
- Writeback – Result is written to the instruction target

*Note: Classic RISC have a fifth stage, memory between exec and writeback, It works on memory operations (LOAD, STORE)

On our custom RISC, this happens on execute. This approach might scale worse, but for our small project will suffice.



Hazard Unit

The Hazard Unit is responsible for detecting conflicts and data dependencies and resolving them.

What is a data dependency / conflict?

Let's look at an example:

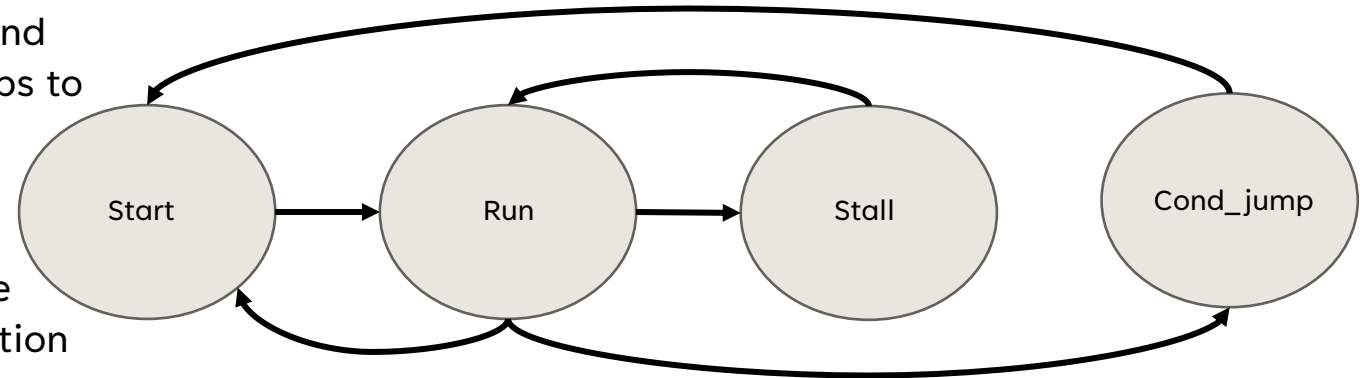
1. Load R1 15
2. Store R1 123

Since our processor can handle 2 instructions a clock, if we try to run both 1 and 2 at the same time, instruction 2 will store the previous value of R1, which is not the intention and will lead to faulty data.

Conflicts like this usually happen when we use the same register for write then read operation
Or the opposite, read then write operation.

Hazard Unit

- Start State – Load initial instructions from the stack.
- Run State – Check if the next **3** instructions have data dependency, and sending the next valid instruction to the CU. (what will happen if we check less than 3 instructions?)
- *Stall State – if a conflict is found, the Hazard Unit will send the CU few (1-3) NO-OP operations to wait for previous ops to finish and resolve the conflict.
- *Conditional Jump State – The Hazard Unit monitors conditional jumps and, if the condition is met, updates the instruction pointer to the target address; otherwise, execution proceeds sequentially.



*Modern CPUs have better tools to deal with conflicts and jumps, [Branch Prediction](#) is a complex topic we won't dive into.

*Modern CPUs have '[shortcuts](#)' in their pipeline, allowing data to skip certain stages to get where they need in time.

ADD R1 R2 R3 EXAMPLE: FETCH

The instruction is approved by the Hazard Unit and gets taken by the Control Unit for further processing

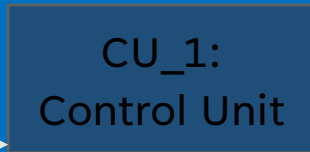
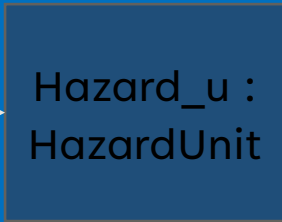
Inst_stack:
RAM

data_bus_out

Hazard_u :
HazardUnit

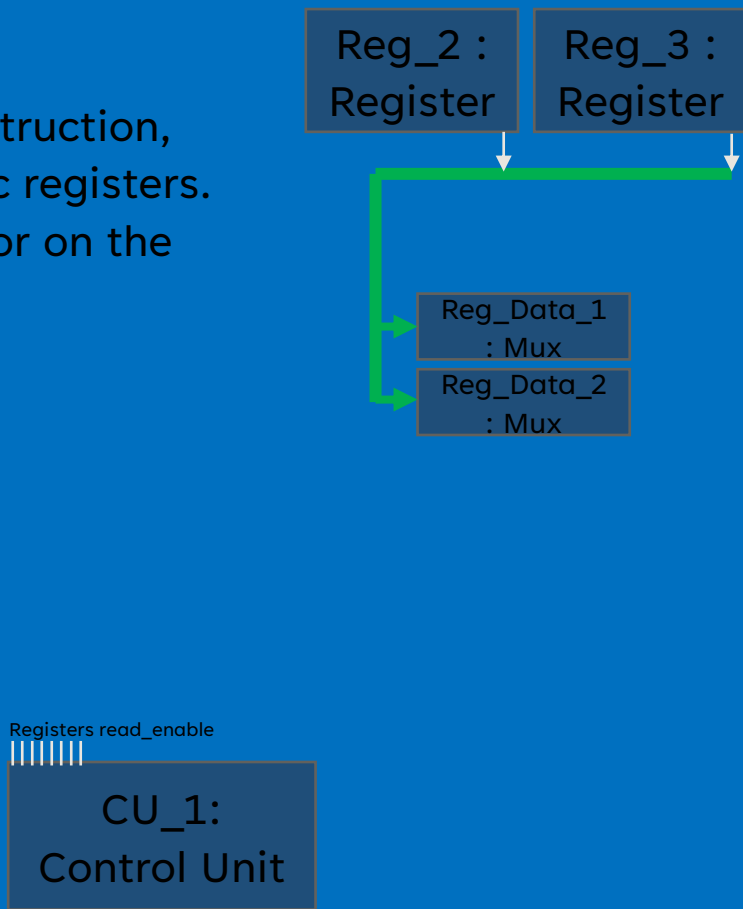
Instructions

CU_1:
Control Unit



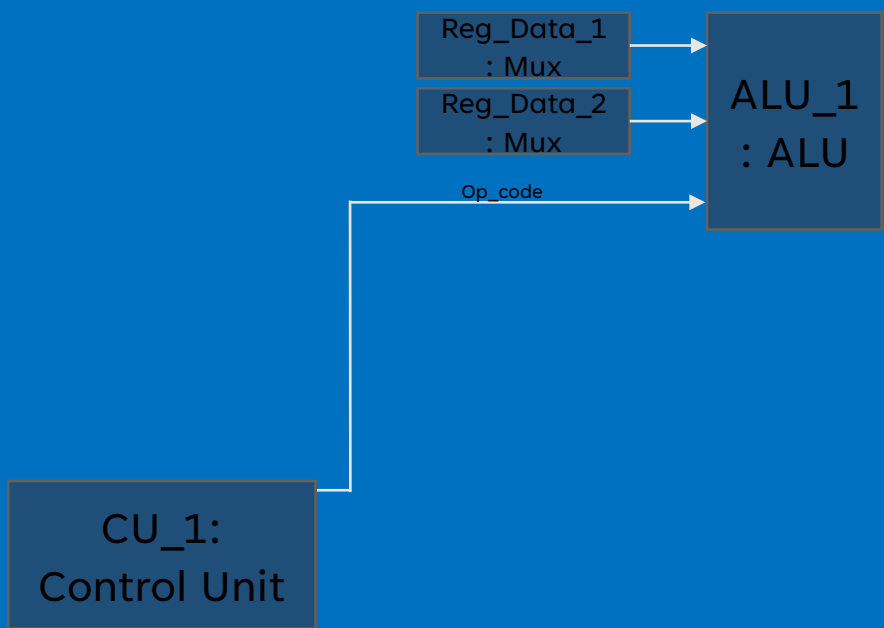
ADD R1 R2 R3 EXAMPLE: DECODE + MEMORY

The Control Unit decodes the instruction, and turns read_enable on the src registers. Also activates the correct selector on the MUXes



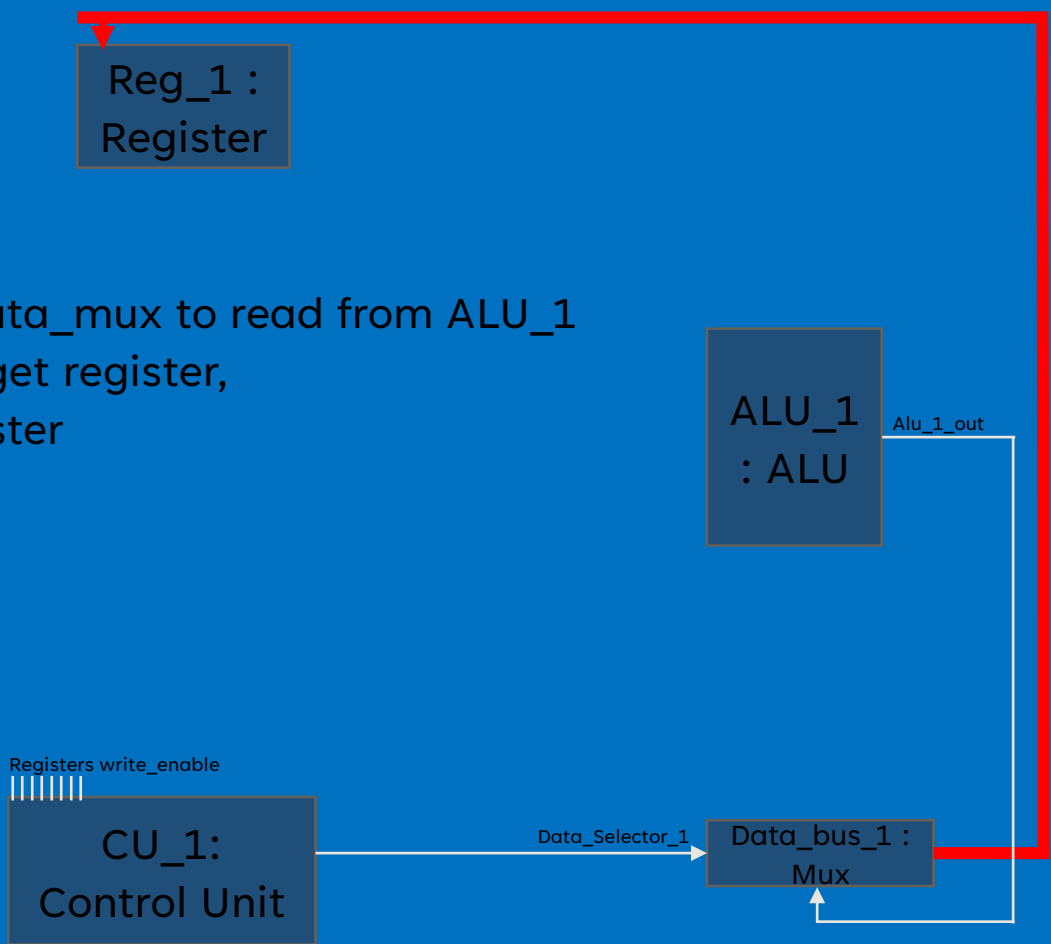
ADD R1 R2 R3 EXAMPLE: EXECUTE

The Control Unit sends the ALU the op code, and the data arrive through the muxes, the ALU processes the operation.



ADD R1 R2 R3 EXAMPLE: WRITEBACK

The Control Unit activates the data_mux to read from ALU_1 output, and write_enable on target register, the data gets written to the register



Few things to notice:

Notice how the Control Unit is in charge on every single step in the pipeline, each time controlling data flow using MUXes and read/write enables.

Also note, that the Control Unit does that on every single clock for different instructions, while EXECUTE happens for one instruction another, different instruction might be in DECODE stage and another one in WRITEBACK stage.

That means that every part of the processor is at work most of the time.

Control Unit

We've now reached the final component of the processor.

As we saw in the example above, the CU's main job is to activate different parts of the CPU at the correct time, Essentially the CU is a box of shift registers.

A shift register is a storage element that “shifts” its data on every clock cycle, either to the left or right. By using chains of these registers, the CU can delay control signals so that specific components.

The CPU utilizes two Control Units, each getting its instructions from the Hazard Unit. Each CU runs independently on received instructions.

Running a program

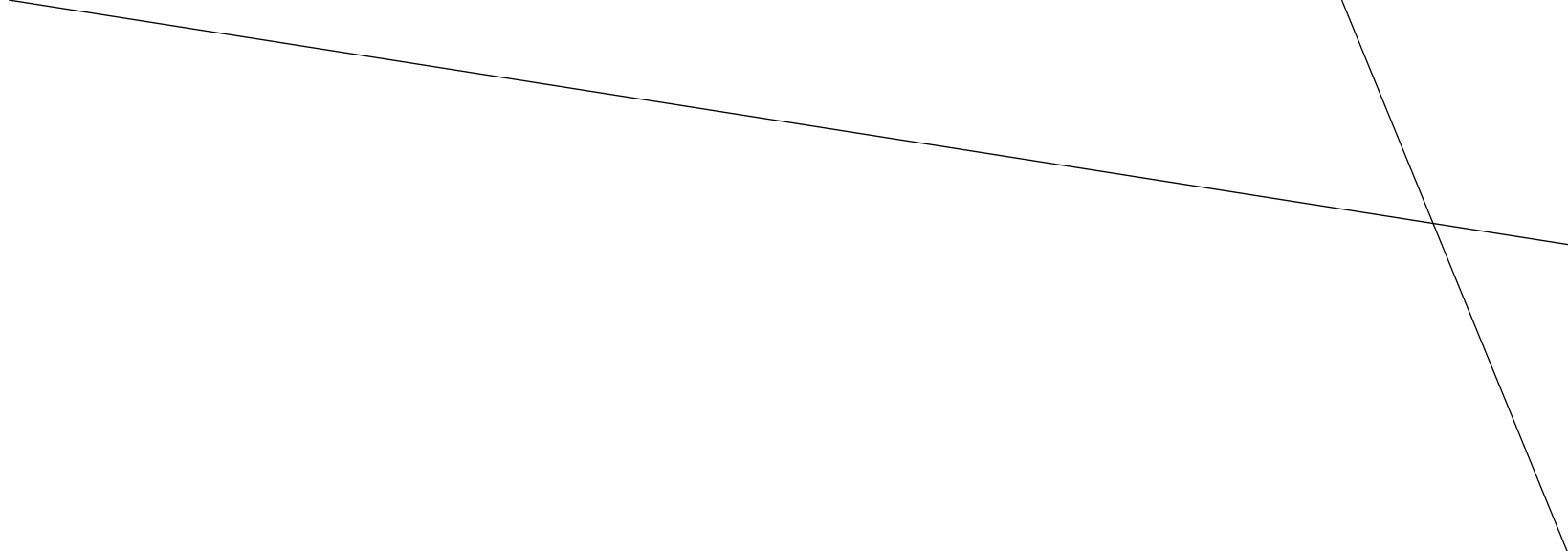
We provide a python script that converts our assembly code into binary for the CPU

For Example:

0011001000010110	LOADI R2 22
0010001000001010	STORE R2 10
0001010000000001	LOAD R4 1
0101000100100100	SUB R1 R2 R4
1110000000000010	JMP 2

In Simulation the stack gets loaded from a text file.

On Embedden systems the stack gets pre-filled, 'burned', usually without the ability to reload it.

Two thin, dark lines intersect in the top right corner of the slide. One line is nearly horizontal, sloping slightly downwards from left to right. The other line is more vertical, sloping downwards from top to bottom.

Thanks for reading!